

# 電腦圖學 Homework #1

## A Drawing Panel

黃俊源 01257128 2026-10-06

### Contents

|                           |          |                                      |           |                                |           |
|---------------------------|----------|--------------------------------------|-----------|--------------------------------|-----------|
| <b>1 專案介紹</b>             | <b>1</b> | <b>5 Fill 與 Hit Test</b>             | <b>10</b> | <b>A Mini GUI Framework</b>    | <b>16</b> |
| 1.1 作業目標                  | 1        | 5.1 Fill Rendering                   | 10        | A.1 設計動機與理念                    | 16        |
| 1.2 Graphoria 系統概述        | 1        | 5.2 Point-in-Polygon                 | 10        | A.2 Element Tree               | 16        |
| <b>2 使用者操作說明</b>          | <b>2</b> | 5.3 Stroke Hit Test                  | 10        | A.3 Layout : Measure 與 Arrange | 16        |
| 2.1 繪圖工具                  | 2        | 5.4 選取順序與選取框                         | 10        | A.4 事件分派                       | 17        |
| 2.2 繪圖屬性與調色盤              | 2        | <b>6 文字與字型 Rendering</b>             | <b>11</b> | A.5 Window、Modal 與生命週期         | 18        |
| 2.3 滑鼠、快捷鍵與 Grid          | 3        | 6.1 FreeGLUT 字型                      | 11        | A.6 Menu 與 Shortcut            | 19        |
| <b>3 幾何表示與互動式繪圖</b>       | <b>4</b> | 6.2 UTF-8 解碼                         | 11        | <b>B GFNT 自訂點陣字型</b>           | <b>19</b> |
| 3.1 圖形的幾何表示               | 4        | 6.3 文字輸入                             | 11        | B.1 設計目的與產生方式                  | 19        |
| 3.2 互動式 Draft             | 5        | 6.4 GFNT 點陣字型                        | 12        | B.2 檔案結構                       | 20        |
| 3.3 Pencil 路徑採樣           | 5        | <b>7 Framebuffer 與 Raster Output</b> | <b>13</b> | B.3 Font Metrics 與字形定位         | 20        |
| <b>4 Stroke Rendering</b> | <b>5</b> | 7.1 Color Buffer 保存與還原               | 13        | B.4 載入與驗證                      | 21        |
| 4.1 用三角形畫粗線               | 5        | 7.2 座標轉換與像素傳輸                        | 13        | B.5 查找與替代字元                    | 21        |
| 4.2 Line Join             | 6        | 7.3 PPM 圖片輸出                         | 14        | <b>C 程式架構</b>                  | <b>22</b> |
| 4.3 Line Cap              | 7        | <b>8 遇到的困難與心得</b>                    | <b>15</b> | C.1 系統架構                       | 22        |
| 4.4 Miter Limit           | 8        | 8.1 Stroke 幾何實作                      | 15        | C.2 資料模型                       | 22        |
| 4.5 退化幾何處理                | 8        | 8.2 事件與介面設計                          | 15        | C.3 繪圖工具                       | 24        |
|                           |          | 8.3 跨平台與文字處理                         | 15        | C.4 繪製一條直線的流程                  | 24        |
|                           |          | 8.4 心得                               | 15        |                                |           |

# 1 專案介紹

## 1.1 作業目標

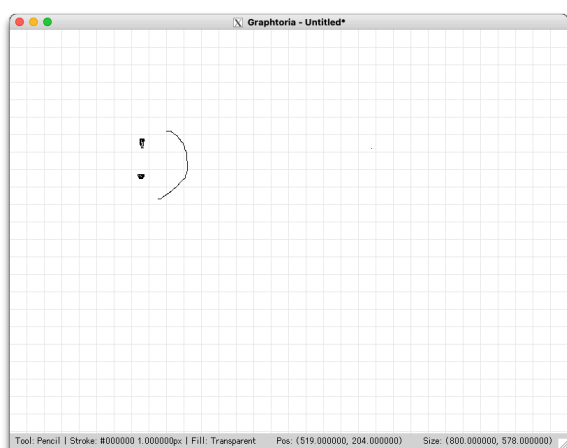
這次作業要用 OpenGL 和 FreeGLUT 做一個可以實際操作的 Drawing Panel。使用者可以用滑鼠在視窗裡畫點、直線、矩形、橢圓、多邊形和自由曲線，也可以輸入文字。除了把圖形畫出來，程式還要處理鍵盤、滑鼠移動、視窗重畫等事件，並讓使用者從選單調整顏色、線寬、點大小、填色方式和字型。

## 1.2 Graphtoria 系統概述

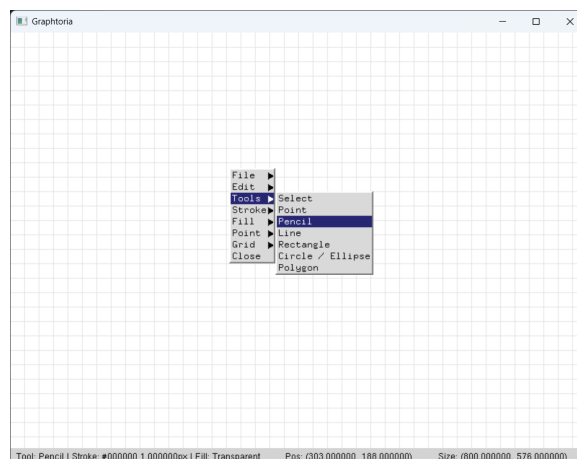
本次實作的程式命名為 **Graphtoria**。系統提供 Select、Point、Pencil、Line、Rectangle、Ellipse、Polygon 和 Text 等工具，主要透過滑鼠在 Canvas 上操作；右鍵選單則用來切換工具，以及設定 Stroke、Fill、Point Size、Font 與 Grid。

繪圖過程中的圖形會先保存為 Draft，讓使用者在完成操作前看到即時預覽。畫面內容則可保存至 Color Buffer，當視窗重新顯示時直接還原；Scene 也能輸出成 PPM P6 圖片，且輸出結果不包含 Grid、狀態列等介面元素。

除了基本繪圖功能，系統亦支援物件選取與刪除、Undo / Redo、多視窗、文字輸入對話框和自訂顏色選擇器。Stroke Rendering 包含 Miter、Bevel、Round Join 與 Butt、Square、Round Cap；文字部分除了 FreeGLUT 內建字型，也支援 UTF-8 與自訂 GFNT 點陣字型。程式內部將繪圖工具、文件內容、事件處理和 Renderer 分開，以避免所有操作狀態集中在 GLUT callback 中。



(a) 主畫面與繪圖結果



(b) 在 Canvas 開啟右鍵選單

Figure 1: Graphtoria 的主畫面與階層式選單

## 2 使用者操作說明

程式啟動後，中間是主要的 Canvas，底部狀態列會顯示目前工具、樣式、滑鼠座標和畫布大小。在 Canvas 按滑鼠右鍵可以開啟階層式選單。

### 2.1 繪圖工具

Table 1: 繪圖工具與操作方式

| 工具               | 操作方式                                                                  |
|------------------|-----------------------------------------------------------------------|
| <b>Select</b>    | 在物件上單擊即可選取；物件重疊時會選到最上層。點擊空白處或按 Esc 可取消選取，Delete 與 Backspace 可刪除選取的物件。 |
| <b>Point</b>     | 在 Canvas 上單擊一次放置方點，大小使用目前設定的 Point Size。                              |
| <b>Pencil</b>    | 按住左鍵並拖曳以繪製自由曲線，程式會沿著滑鼠路徑取樣；放開左鍵後完成。                                   |
| <b>Line</b>      | 從起點拖曳到終點後放開；拖曳時按住 Shift，可將方向限制為水平、垂直或 45°。                            |
| <b>Rectangle</b> | 從一個角拖曳到對角，放開左鍵後完成；按住 Shift 會限制為正方形。                                   |
| <b>Ellipse</b>   | 拖曳決定橢圓的外接矩形，放開左鍵後完成；按住 Shift 會限制為正圓。                                  |
| <b>Polygon</b>   | 每次單擊加入一個頂點，移動滑鼠時會預覽下一條邊。雙擊或按 Enter 完成，Backspace 移除上一點。                |
| <b>Text</b>      | 先在 Canvas 上單擊文字的起始位置，再於對話框輸入內容；確認後建立文字物件。                             |

### 2.2 繪圖屬性與調色盤

顏色分為 Primary Color 和 Fill Color 兩種。Primary Color 用於線條、點和文字；Fill Color 則是封閉圖形內部的填色<sup>1</sup>。兩種顏色都可以從選單的調色盤挑選常用顏色，或開啟圖 2 的顏色選擇器，透過色彩區域、Hue slider、RGB 數值或 Hex 色碼指定任意顏色。

Fill 有三種模式：Outline 只畫外框；Filled 以 Primary Color 填滿整個圖形；Advanced 則以 Fill Color 填滿內部，再用 Primary Color 畫外框，可以讓內外使用不同顏色。Stroke 可以設定線寬、轉角樣式（Miter、Bevel、Round Join）和端點樣

<sup>1</sup>Fill Color 與 Stroke 的轉角、端點樣式只在 Advanced 模式下生效。

式（Butt、Square、Round Cap）<sup>1</sup>。另外也可以調整 Point Size 和文字字型。

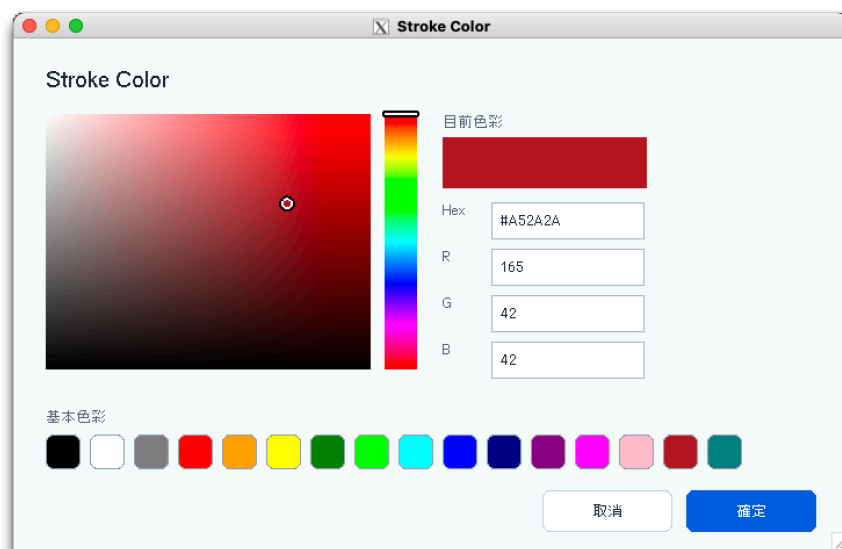


Figure 2: 顏色選擇器，可用色彩區域、RGB 或 Hex 設定顏色

## 2.3 滑鼠、快捷鍵與 Grid

滑鼠左鍵用來繪圖和選取，右鍵開啟選單。程式中的 Primary 在 macOS 代表 Command，在 Windows/Linux 代表 Ctrl。所有可用快捷鍵與情境按鍵如下：

Table 2: Graphtoria 的完整快捷鍵

| 快捷鍵             | 功能                               |
|-----------------|----------------------------------|
| Primary+Z       | Undo；若正在繪圖，會先取消目前的 Draft。        |
| Primary+Shift+Z | Redo。                            |
| Primary+N       | 建立新文件。                           |
| Primary+Shift+N | 建立新的繪圖視窗。                        |
| Primary+O       | 開啟文件。                            |
| Primary+S       | 儲存目前文件。                          |
| Primary+Shift+S | 另存新檔。                            |
| Primary+E       | 將目前 Scene 輸出為 PPM P6 圖片。         |
| Primary+R       | 重新顯示快取畫面，並重設鍵盤與滑鼠輸入狀態。           |
| C               | 清空目前畫布。                          |
| G               | 依序切換 Lines、Dots、None 三種 Grid 模式。 |
| 0               | 切換為 Select 工具。                   |
| 1               | 切換為 Pencil 工具。                   |
| 2               | 切換為 Line 工具。                     |
| 3               | 切換為 Rectangle 工具。                |
| 4               | 切換為 Ellipse 工具。                  |

| 快捷鍵     | 功能                                          |
|---------|---------------------------------------------|
| 5       | 切換為 Polygon 工具。                             |
| 6       | 切換為 Text 工具。                                |
| [       | Stroke Width 與 Point Size 減少 1 px，最小為 1 px。 |
| ]       | Stroke Width 與 Point Size 增加 1 px。          |
| Shift+[ | Stroke Width 與 Point Size 減少 5 px，最小為 1 px。 |
| Shift+] | Stroke Width 與 Point Size 增加 5 px。          |

Grid 有 Lines、Dots 和 None 三種模式。它只是在 Scene 底下顯示的輔助背景，不會出現在輸出圖片中。

Text 工具和未儲存提示會開啟程式自製的 Modal Dialog。對話框存在時，背景視窗不再接收輸入，等使用者確認或取消後才恢復。圖 3 是程式目前使用的兩種對話框。



Figure 3: Graphtoria 的 Modal Dialog

## 3 幾何表示與互動式繪圖

### 3.1 圖形的幾何表示

不同工具取得圖形的方法不一樣，但完成繪圖後都會將 Line、Rectangle、Path 和 Polygon 都整理成頂點序列，另外記錄路徑是否封閉。Ellipse 則先由外接矩形算出中心和半徑，再取樣成 64 個頂點。Point 只需要位置；Text 會保存基準位置、UTF-8 字串和文字樣式。

這樣做的好處是 Renderer 和 Select Tool 都可以使用同一份幾何資料，不需要知道頂點原本是用拖曳、連續移動還是多次點擊產生的。

### 3.2 互動式 Draft

使用者在拖曳 Line 或 Rectangle 時，圖形還沒正式完成，但畫面上仍要即時預覽。這種暫時圖形稱為 Draft。滑鼠移動只會更新 Draft，Canvas 再把它畫在原本的 Scene 上面；等到使用者放開滑鼠或完成 Polygon，才把 Draft 轉成正式物件並加入 Document。

這個做法也讓 Undo 比較單純。一次拖曳雖然會收到很多 Motion event，最後只建立一筆 Command；如果繪圖途中按 Undo，程式會先取消目前 Draft，而不是刪除上一個已完成物件。

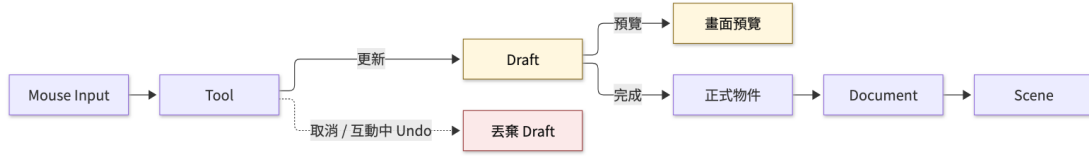


Figure 4: Draft 從滑鼠輸入、預覽到提交成正式物件的流程

### 3.3 Pencil 路徑採樣

Pencil 需要一直收集滑鼠位置，但如果把每個 Motion event 都存下來，會產生很多幾乎重合的點。設上一個採樣點為  $P_{i-1}$ 、目前位置為  $P$ 、線寬為  $w$ ，最小距離設為

$$d_{\min} = \max(0.2w, 1 \text{ px}).$$

只有  $\|P - P_{i-1}\| \geq d_{\min}$  時才加入新點。除了減少資料量，這也能避免很短的線段讓後面的法向量和轉角計算變得不穩定。

## 4 Stroke Rendering

### 4.1 用三角形畫粗線

Outline 與 Filled 模式以 `glLineWidth()` 搭配 OpenGL 線條繪製。線寬較小時不易察覺問題，但線寬增加到 25 px 時，矩形轉角會出現圖 5a 左側的缺角與突出。原因是 OpenGL 的寬線逐段獨立光柵化，線段兩端只會切成水平或垂直的邊，相鄰兩段在轉角處無法對齊。此外，OpenGL 無法設定轉角與端點樣式，最大線寬也由實作決定。因此 Advanced 模式改為自行將中心線展開成三角形，結果如同圖右側。

展開的基本單位是單一線段。對線段  $A \rightarrow B$ ，設線寬為  $w$ 、半線寬  $h = w/2$ ，其單位方向與法向量為

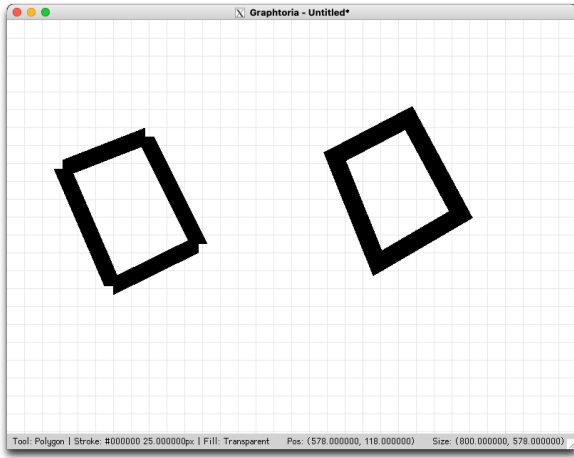
$$\vec{u} = \frac{B - A}{\|B - A\|}, \quad \vec{n} = (-u_y, u_x).$$

由於 Canvas 的 Y 軸向下， $\vec{n}$  指向前進方向的右側，因此線段兩端的左右邊界點為

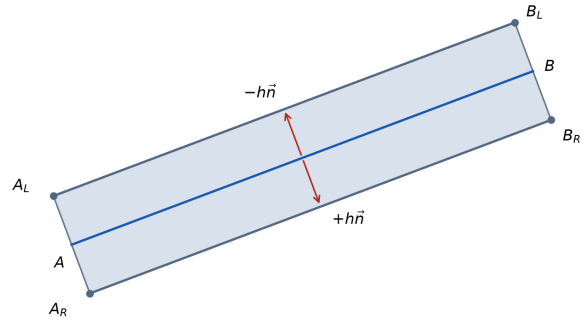
$$A_L = A - h\vec{n}, \quad A_R = A + h\vec{n}, \quad B_L = B - h\vec{n}, \quad B_R = B + h\vec{n}.$$

這四點圍成寬度為  $w$  的四邊形，再拆成  $\triangle A_L A_R B_R$  和  $\triangle A_L B_R B_L$  兩個三角形交由 OpenGL 繪製。

然而折線的每個頂點同時屬於前後兩段，若各段分別使用  $B \pm h\vec{n}$ ，轉角仍無法對齊。因此程式先為每個頂點計算左右兩個「截面點」，再以相鄰頂點的截面點組成線身。截面點預設為上述邊界點，在轉角處依第 4.2 節的 Join 調整，在開放路徑的端點則依第 4.3 節的 Cap 調整。長度為零的線段沒有方向，不產生線身，相關處理見第 4.5 節。



(a) OpenGL 粗線與自製 Stroke



(b) 中心線向兩側展開

Figure 5: 自行建立 Stroke Geometry 的原因與基本方法

## 4.2 Line Join

若每段僅依圖 5b 各自展開，轉角外側會留下缺口，內側則互相重疊，因此每個頂點都需要另外處理。對每個頂點  $B$ ，取前一點  $A$  與下一點  $C$ ，兩段的方向與法向量分別為

$$\vec{u} = \frac{B - A}{\|B - A\|}, \quad \vec{v} = \frac{C - B}{\|C - B\|}, \quad \vec{n}_0 = (-u_y, u_x), \quad \vec{n}_1 = (-v_y, v_x).$$

與第 4.1 節相同， $AB$  的左、右邊界是中心線平移  $-h\vec{n}_0$  與  $+h\vec{n}_0$  所得的 offset line， $BC$  亦同。

**判斷內外側** 處理轉角前需先確定內側與外側。兩段的轉向由二維外積判斷：

$$c = \vec{u} \times \vec{v} = u_x v_y - u_y v_x.$$

在 Y 軸向下的畫面上， $c > 0$  表示向右（順時針）轉，此時右側為內側、左側為外側； $c < 0$  時相反。 $c$  接近 0 表示兩段幾乎平行或折返，於第 4.5 節說明。



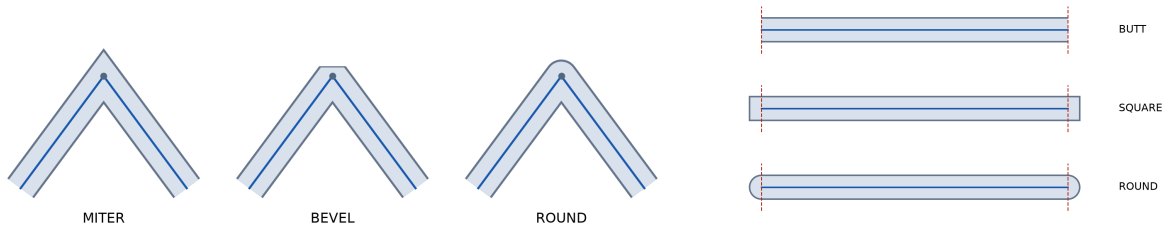
**求 offset line 交點** 左右兩側分別求  $AB$  與  $BC$  兩條 offset line 的交點。將兩條直線表示為  $P + t\vec{r}$  與  $Q + s\vec{w}$ ，當  $\vec{r} \times \vec{w} \neq 0$  時，交點為

$$t = \frac{(Q - P) \times \vec{w}}{\vec{r} \times \vec{w}}, \quad X = P + t\vec{r}.$$

為避免除以接近零的數，當  $|\vec{r} \times \vec{w}| \leq \varepsilon \|\vec{r}\| \|\vec{w}\|$  時視為兩線平行，不使用交點。以下將內側交點記為  $X_{\text{in}}$ 、外側交點記為  $X_{\text{out}}$ 。

**三種 Join** 三種 Join 的內側處理相同：兩段的內側邊界都收至  $X_{\text{in}}$ ，使線身在內側相接而不重疊。差別在於外側缺口的補法，效果如圖 6a。

- **Miter**：外側同樣收至  $X_{\text{out}}$ ，兩段線身在尖角處直接接合，不需額外三角形。轉角很尖時  $X_{\text{out}}$  會遠離頂點，因此超過 Miter Limit 時改用 Bevel，推導見第 4.4 節與圖 7。
- **Bevel**：外側保留兩段各自在  $B$  的截面端點  $O_0$ 、 $O_1$ （向右轉時為  $B - h\vec{n}_0$  與  $B - h\vec{n}_1$ ），再補上  $\triangle O_0O_1X_{\text{in}}$ ，將尖角切平。
- **Round**：先用  $\triangle X_{\text{in}}BO_0$  和  $\triangle X_{\text{in}}O_1B$  填滿頂點與內側之間的區域，再以  $B$  為圓心、 $h$  為半徑，從  $O_0$  到  $O_1$  繪製 triangle fan。圓弧的旋轉方向依  $c$  的正負決定，以確保畫在外側。分段數為  $\max\left(4, \lceil \frac{|\Delta\phi|}{(\pi/16)} \rceil\right)$ ，其中  $\Delta\phi$  為圓弧張角，張角越大分段越多。



(a) Miter、Bevel、Round Join

(b) Butt、Square、Round Cap

Figure 6: 程式支援的 Line Join 與 Line Cap

### 4.3 Line Cap

開放路徑的第一點沒有前一點、最後一點沒有下一點，程式將缺少的鄰點設為頂點本身。這兩個頂點只有一個方向，因此不計算 Join，改依設定繪製 Cap。圖 6b 的紅色虛線為原始端點位置，三種 Cap 的差別如下：

- **Butt**：截面  $B \pm h\vec{n}$  即為線身盡頭，邊界停在端點上。



- **Square**：將端點截面沿線段方向向外推  $h$ ，起點朝  $-\vec{v}$ 、終點朝  $+\vec{u}$ 。由於只移動線身四邊形的頂點，延伸部分不需額外三角形。
- **Round**：以端點為圓心、 $h$  為半徑，從截面一側至另一側繪製半圓 triangle fan；起點的半圓位於線段後方，終點的半圓位於前方。

封閉路徑 (Rectangle、Ellipse、Polygon) 取鄰點時首尾相接，第一點的前一點即為最後一點，因此每個頂點都套用 Join，不會出現 Cap。若路徑只有一個頂點 (例如 Pencil 僅點擊一次)，則直接依 Cap 繪製：Round 為直徑  $w$  的圓點，Square 為邊長  $w$  的正方形，Butt 不繪製任何內容。

#### 4.4 Miter Limit

Miter 在很尖的轉角會拉得很長。設中心線夾角為  $\theta$ 、半線寬為  $h$ ，原頂點至 Miter 尖端的距離為  $m$ 。角平分線會把夾角分成  $\theta/2$ ，而 offset 邊界至中心線的垂直距離正好是  $h$ ，因此由直角三角形可得

$$\sin\left(\frac{\theta}{2}\right) = \frac{h}{m}, \quad m = \frac{h}{\sin(\theta/2)}.$$

Miter Limit  $L$  定義為允許的最大 Miter 長度與半線寬之比，所以條件為

$$\frac{m}{h} \leq L.$$

代入前式後可得到

$$\frac{1}{\sin(\theta/2)} \leq L \iff \theta \geq 2 \arcsin\left(\frac{1}{L}\right).$$

因此  $L$  越大，允許的最小夾角越小，Miter 也可能拉得越長。程式預設  $L = 4$ ，對應的最小夾角約為  $29^\circ$ ；不符合條件時改用 Bevel，避免出現過長尖刺。

#### 4.5 退化幾何處理

第 4.2 節的做法假設每個頂點前後兩段都有明確方向，且轉角夾角合理。實際以 Pencil 繪製時，路徑經常出現重複點、極短線段、幾乎共線或接近  $180^\circ$  的折返。這些情況下，單位方向或 offset line 交點在數值上不穩定，若直接套用一般公式，容易產生跨越畫面的尖刺或錯誤的三角形，因此程式針對以下三種情況另外處理。

**重複點與零長度線段** 零長度線段沒有方向，無法計算法向量。第 3.3 節的取樣距離已在輸入階段避免大部分過近的點；繪製前，程式再移除與前一點座標差小於  $\epsilon$  的連續重複點。之後若頂點與前一點或下一點仍重合，就視為該側沒有線段：不產生線身，也不計算 Join，與第 4.3 節的端點採用相同處理。

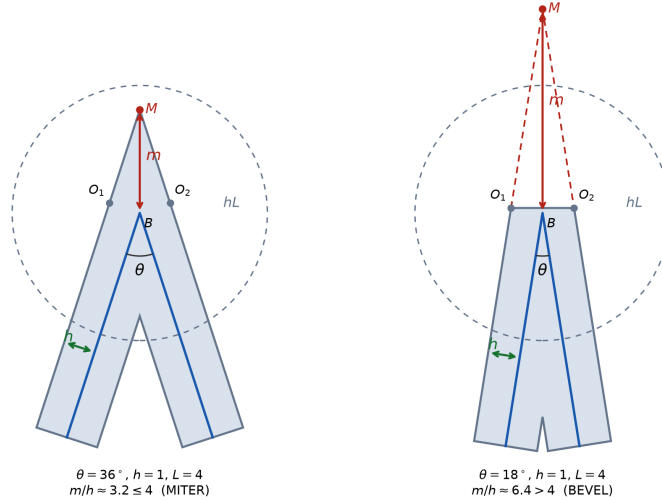


Figure 7: Miter 長度在限制內與超過限制時的處理

**近平行與折返** 當  $|c| \leq \varepsilon$  時，兩段幾乎平行，offset line 交點不存在或極不穩定，此時改以內積  $d = \vec{u} \cdot \vec{v}$  區分兩種情況：

- $d > 0$ ，幾乎同方向：頂點實際上沒有轉彎，不需要 Join。 $\vec{n}_0$  與  $\vec{n}_1$  理論上相同，程式取兩段截面點的平均值作為共用截面，以減少浮點誤差造成的細縫。
- $d < 0$ ，接近  $180^\circ$  折返：路徑原地掉頭，內側與外側的概念已經退化，Miter 交點也不存在。程式以頂點  $B$  作為內側點，以進入線段在  $B$  的兩個截面端點作為外側點，設定為 Round 時補上半圓，否則一律使用 Bevel，使折返處成為平整或圓弧的端面。

另外，即使  $|c| > \varepsilon$ ，若 offset line 求交仍因數值問題失敗，程式同樣改用 Bevel 或 Round，並以  $B$  作為內側點。

**內側交點過遠** 第 4.4 節的 Miter Limit 只限制外側交點。內側交點與頂點的距離同樣是  $h/\sin(\theta/2)$ ，轉角越尖就越遠；若相鄰線段很短，內側交點甚至會落在線段範圍之外，此時把內側邊界收到該點，三角形會翻折並向外突出。因此程式以

$$\|X_{\text{in}} - B\| \leq \max(2h, \min(\ell_{AB}, \ell_{BC}) + h)$$

作為內側交點的合理範圍，其中  $\ell_{AB}$ 、 $\ell_{BC}$  為前後兩段的長度。超出範圍時改以頂點  $B$  作為內側點，雖然內側會有少量重疊，但能避免短線段產生延伸到畫面遠處的三角形。

## 5 Fill 與 Hit Test

### 5.1 Fill Rendering

Fill 只作用於封閉圖形，依第 2.2 節的模式繪製：Outline 只畫外框；Filled 以 Primary Color 填滿；Advanced 先以 Fill Color 填滿內部，再疊上第 4.1 節的自製 Stroke，如圖 13 的房屋與太陽。填色以 GL\_POLYGON 送出頂點，而 OpenGL 只保證其對凸多邊形正確；Rectangle 與 Ellipse 不受影響，但 Polygon 工具畫出的凹多邊形可能填錯，完整支援需先 triangulation。

### 5.2 Point-in-Polygon

有可見填色的封閉圖形，點擊內部也應能選取。判斷採用 Ray Casting：從測試點  $(x, y)$  向右發射水平射線，與邊界交點數為奇數即在內部。對每條邊  $(x_i, y_i) \rightarrow (x_j, y_j)$ ，只有  $(y_i > y) \neq (y_j > y)$  時射線才會穿過，交點為

$$x_c = x_i + \frac{(x_j - x_i)(y - y_i)}{y_j - y_i},$$

若  $x_c > x$  便切換一次 inside 狀態。此條件只計入  $y \in [\min(y_i, y_j), \max(y_i, y_j))$  的邊，射線通過共同頂點時只計算一次，水平邊也會被略過而不會除以零。

### 5.3 Stroke Hit Test

點擊線條時，計算測試點  $P$  到線段  $AB$  的最短距離。將  $P - A$  投影到  $B - A$  得到參數  $t$ ：

$$t = \text{clamp}\left(\frac{(P - A) \cdot (B - A)}{\|B - A\|^2}, 0, 1\right), \quad Q = A + t(B - A).$$

其中  $\text{clamp}(x, 0, 1)$  將  $x$  限制在  $[0, 1]$  之間，使投影點  $Q$  落在線段  $AB$  上，而非其延長線上。若  $\|P - Q\| \leq \tau = \max(w/2 + 3, 5)$  px 即命中，其中  $w/2$  對應線身半寬，其餘為點擊容差； $A = B$  時直接取  $\|P - A\|$ 。封閉路徑另外檢查首尾相連的邊，Point 則以 Point Size 代替  $w$ 。

### 5.4 選取順序與選取框

封閉且有可見填色的 Shape 先做第 5.2 節的判定，未命中再做第 5.3 節的判定；Text 則以字型 metrics 求出外接矩形。Scene 依陣列順序繪製，後加入的物件在上層，因此 Select 工具從尾端往前檢查，第一個命中的即為最上層物件。選取框為頂點的外接矩形向外擴張  $\max(w/2, 2)$  px，避免壓在線條上。

## 6 文字與字型 Rendering

### 6.1 FreeGLUT 字型

文字物件保存左上角位置、UTF-8 字串與字型樣式。FreeGLUT 內建兩類字型：Bitmap Font 以 `glRasterPos2f()` 指定位置後呼叫 `glutBitmapCharacter()`，由於 raster position 對應字元基線附近，程式依各字型的高度與 origin 偏移，將「行頂端」換算為基線位置；Stroke Font 以線段描述字元，座標 Y 軸向上，繪製前先平移到基線，再以 `glScalef(s, -s, 1)` 縮放並翻轉 Y 軸以配合 Canvas。兩者都逐 byte 繪製，只涵蓋 ASCII，無法顯示中文。

### 6.2 UTF-8 解碼

為了支援中文，字串在繪製與編輯前都先由 UTF-8 解碼為 Unicode code point，之後的游標移動、字形查找都以 code point 為單位。不合法的 byte 序列一律轉為替代字元 U+FFFD，且只略過 1 byte，讓後續合法的字元仍能正確解碼。

### 6.3 文字輸入

UTF-8 的一個字元可能佔 1 到 4 bytes，例如「你」編碼為 E4 BD A0 三個 bytes。若直接在 `std::string` 上以 byte 移動游標或刪除，游標可能停在字元中間，如圖 8 下排的紅線，此時 Backspace 會留下無法解碼的殘缺 bytes。因此輸入框以 code point 陣列作為編輯資料，游標只會停在圖中上排的字元邊界，Backspace 與 Delete 一次刪除一個完整字元；內容改變後才重新編碼為 UTF-8 字串供外部使用。



Figure 8: 同一字串的 code point 與 UTF-8 bytes 對照。以 byte 為單位移動游標可能切斷多 byte 字元，因此輸入框以 code point 為單位編輯

FreeGLUT 的鍵盤 callback 每次只提供一個 byte，無法接收輸入法組字的結果。為此輸入框提供 Unicode 輸入模式：輸入反引號 ` 後接 4 位十六進位數字，即可插入對應的 code point，例如「`4F60」為「你」。無效的 code point 同樣會轉為 U+FFFD。

## 6.4 GFNT 點陣字型

FreeGLUT 沒有中文字形，而 TrueType 以 Bézier 曲線描述字形，解析與光柵化的成本都高，因此本專案另外設計 GFNT<sup>2</sup> 點陣字型格式，預先將字形轉為 alpha bitmap。檔案包含字型 metrics、依 code point 排序的 Glyph 表與 bitmap 資料，詳細格式見附錄 B。

繪製流程如圖 9。繪製時維護一個「pen」，即下一個字元在基線上的起始位置，初始為文字的左端。對每個 code point，先在依 code point 排序的 Glyph 表中以二分搜尋（std::lower\_bound）找到 Glyph，找不到時依序改用 U+FFFD 與問號；再由 bearing 求出 bitmap 左上角相對於 pen 的位置，畫完後將 pen 向右移動 advance，遇到換行則回到左端並下移一個行高。由於 glDrawPixels() 不會以目前的 glColor 替單通道 bitmap 上色，程式先建立 RGBA buffer：RGB 填入文字顏色，A 為 bitmap alpha 乘上文字顏色的 alpha，再開啟 blending 繪製。GFNT 的 bitmap 由上往下儲存，與 OpenGL 預設的列順序相反，因此將 glPixelZoom() 的 Y 設為 -1。實際輸出如圖 10。

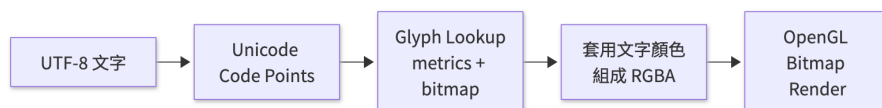


Figure 9: UTF-8 字串到 GFNT 點陣字形的繪製流程

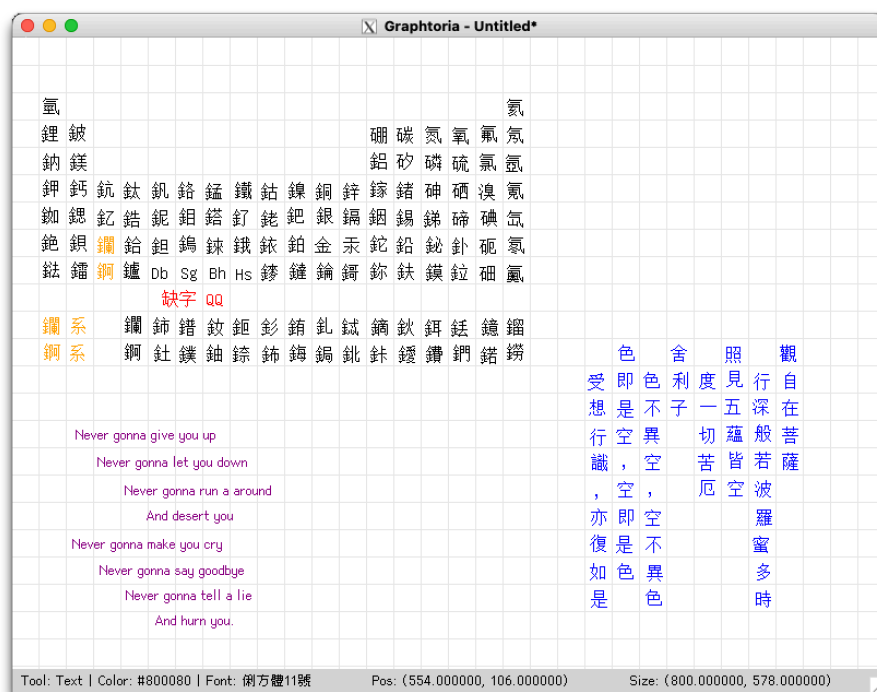


Figure 10: GFNT 中文字型在 Graphtoria 中的實際輸出

<sup>2</sup>本專案的自定義格式，取自“Graphtoria Font”

## 7 Framebuffer 與 Raster Output

### 7.1 Color Buffer 保存與還原

程式以 GLUT\_SINGLE 直接繪製在 front buffer。視窗重新出現時系統會要求重畫，但 Scene 往往沒有改變，因此每個 Window 以 ColorBuffer 保存最後一次完整畫面的 RGBA 像素，並以 contentDirty（內容已改變，需重新 render）與 needsCapture（已 render、尚未擷取）兩個狀態決定重畫方式。如圖 11，display callback 在內容改變、尚未擷取或尺寸不符時重新 render，否則以 glDrawPixels() 直接還原。擷取則交給每 16 ms 執行一次的 timer，確認內容沒有再次改變後才以 glReadPixels() 讀回，避免擷取到未完成的畫面。

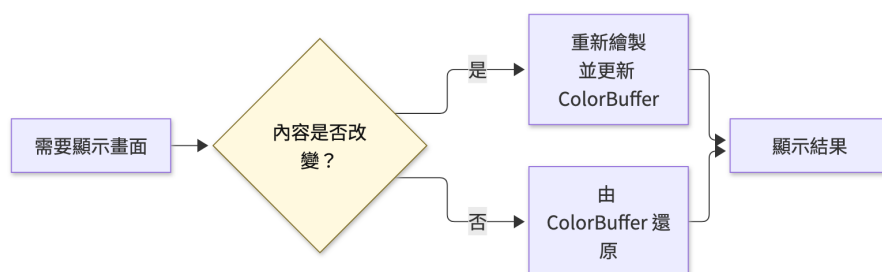


Figure 11: display callback 依內容是否改變，選擇重新 render 或由 ColorBuffer 還原

修改 Scene 的操作會標記 contentDirty；視窗改變大小時重新配置 ColorBuffer；視窗被隱藏時也會標記 contentDirty，因為 OpenGL 不保證被遮住區域的 front buffer 內容仍然有效。表 2 的 Primary+R 則只要求重畫，會直接還原 ColorBuffer。

### 7.2 座標轉換與像素傳輸

UI 座標以左上角為原點、Y 軸向下，OpenGL 的視窗座標則以左下角為原點、Y 軸向上。對高度為  $H$  的視窗中左上角為  $(x, y)$ 、高度為  $h$  的區域，其 OpenGL 底邊為  $y_{\text{bottom}} = H - y - h$ ，如圖 12。讀回的像素由下往上逐列排列，還原時以相同順序寫回，因此 ColorBuffer 內部不需翻轉；讀寫前也將 pack/unpack alignment 設為 1，使每列像素緊密排列。

還原時，glRasterPos2f() 的座標會經過矩陣轉換，不能直接傳入視窗座標。程式先將矩陣設為單位矩陣，以 glRasterPos2f(0, 0) 將 raster position 設在 viewport 中央，再以不含點陣資料的 glBitmap() 位移到目標位置，只需 OpenGL 1.1 而不依賴 glWindowPos()。



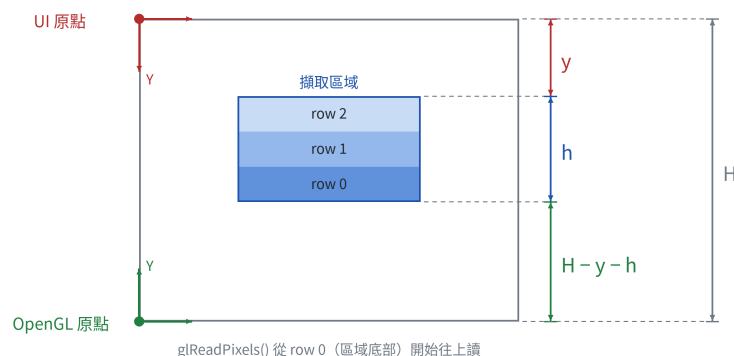


Figure 12: 同一區域在 UI 座標中由頂端量起（紅），在 OpenGL 中由底端量起（綠）；glReadPixels() 由區域底部逐列往上讀取

### 7.3 PPM 圖片輸出

主視窗的 ColorBuffer 包含狀態列與 Grid，不能直接輸出。程式另外建立一個與 Canvas 同尺寸、專供匯出使用的視窗，其中只放一個關閉 Grid 的 Canvas，重用相同的 Renderer 繪製 Scene 後擷取。PPM P6 由 header P6 W H 255 與連續的 RGB bytes 組成，像素由頂端逐列往下存放；ColorBuffer 的列順序相反，因此寫檔時從最後一列往前輸出，並捨棄 alpha。實際輸出如圖 13，方向正確且不含 Grid 與狀態列。

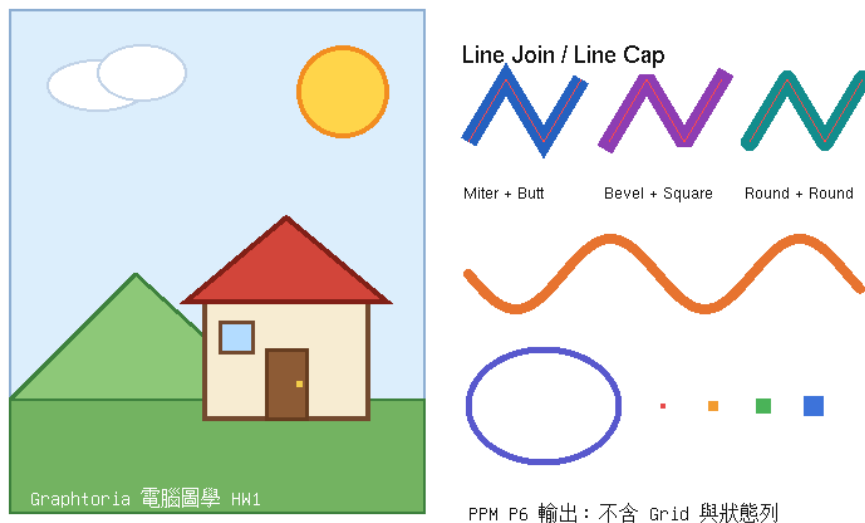


Figure 13: 程式實際輸出的 PPM P6 圖片（720 × 440，轉為 PNG 後放入報告）



## 8 遇到的困難與心得

### 8.1 Stroke 幾何實作

粗線的做法前後改了好幾次。我一開始直接使用原生的 `glLineWidth()`，但線寬調大後，折線轉角處會出現缺角，如圖 5a 左側。為了補上缺口，我先在每個頂點額外畫一個 `GL_POINTS`，但 OpenGL 的點是方形的，轉角處反而多出一塊突出的方角。接著改成在頂點畫圓，外觀雖然接近圓角，但線段與圓會在頂點處重疊。當時我預期之後可能會研究半透明顏色，若真的支援，重疊區域就會被混色兩次，使轉角顏色明顯變深。

因此我最後決定自己計算轉角的幾何：判斷內外側、求 offset line 交點，只補上缺口所需的三角形。做完後上網查資料，才知道這個技術叫 Line Join，Miter、Bevel、Round 正是常見的三種做法，於是照此整理成第 4.2 節的實作；之後快速畫 Pencil 出現的尖刺，則再以 Miter Limit 與內側距離限制解決（第 4.5 節）。

### 8.2 事件與介面設計

FreeGLUT 只提供低階的 callback，沒有 Focus、Mouse Capture、事件冒泡與 Modal。加入多視窗、文字輸入與對話框後，滑鼠拖出 Canvas、背景視窗仍收到輸入等問題陸續出現，我只好逐步建立 Element Tree、Hit Test、Bubbling、Focus 與 Mouse Capture（附錄 A）。這是整份作業花最多時間的部分。

### 8.3 跨平台與文字處理

最難發現的跨平台問題是 Windows 上的 Ctrl+ 字母：FreeGLUT 傳入的是 control code 1 到 26 而非 'a' 到 'z'，導致 Undo、Save 等快捷鍵全部失效。最後以 `mapCharacterWithCtrl()` 還原按鍵，並用 Primary 統一 Command 與 Ctrl。文字方面，FreeGLUT 無法顯示中文，鍵盤 callback 也接收不到輸入法的結果，這兩個限制帶出了 GFNT 字型與 Unicode 輸入模式（第 6.3、6.4 節）。

### 8.4 心得

原本以為這次作業主要是用 OpenGL 把圖形畫出來，實際上最花時間的反而是 OpenGL 沒有提供的部分：GUI、事件傳遞、Stroke 幾何與中文字型。功能陸續增加後，若把工具狀態、文件修改與 OpenGL 指令都寫在 callback 裡，程式很快就難以維護，因此我將 Window/Element、Tool、Document/Command 與 Renderer 分開，這讓我體會到，看似簡單的 Drawing Panel 背後其實需要清楚的模組分工，才能持續除錯與擴充。

## A Mini GUI Framework

### A.1 設計動機與理念

本專案最初直接在 FreeGLUT callback 中處理滑鼠、鍵盤與視窗事件。隨著功能增加，Canvas、狀態列、文字輸入框、對話框與多視窗各自需要排列方式、事件處理與輸入狀態，若全部集中在 callback 或主視窗中，程式很快就難以維護。因此實作過程中逐步抽出 Window、Element、Layout 與 Event 等基礎機制，形成一套只涵蓋本作業需求的 Mini GUI Framework。它並非要重做完整的 GUI toolkit，而是將 FreeGLUT 的低階 callback 與應用程式邏輯分離，讓主視窗、Canvas、輸入框與對話框建立在同一套介面與事件模型上。

設計概念受到瀏覽器中 HTML 與 DOM 的啟發：介面由具有父子關係的節點組成，如同 HTML 文件的元素樹；滑鼠事件先經 Hit Test 找到 target，再沿父節點向上 bubbling；Focus、Mouse Capture 與 Modal 則用來處理實際互動時的輸入狀態。由於沒有 CSS 與瀏覽器排版引擎，版面配置改採較簡單的 Measure/Arrange 兩階段計算，並只提供本作業需要的 Stack 與 Dock 兩種容器。所有元件最後都以 OpenGL 繪製在所屬視窗的 framebuffer 上。

### A.2 Element Tree

介面由 Element 組成樹狀結構。每個 Element 以 `std::unique_ptr` 擁有子元素，因此移除父元素時，整棵子樹會一併釋放。Element 的 bounds 以父元素為座標原點；繪製時遞迴地先平移到自己的左上角，再依序畫背景、自身內容、所有子元素，最後畫游標等前景效果，因此每個元件都只需處理自己的局部座標。Hit Test 也沿著樹往下進行，並從最後一個子元素開始檢查，使後加入、畫在上層的元素優先命中。

目前提供的元件包括 RootElement、DockPanelElement、StackPanelElement、TextElement、InputElement、ButtonElement、CanvasElement，以及顏色選擇器使用的 ColorFieldElement 與 HueSliderElement。主繪圖視窗的結構如圖 14。

### A.3 Layout : Measure 與 Arrange

版面配置分成兩個遞迴階段。**Measure** 由父元素傳入可用大小（任一維度可為「不限制」），元素扣除 margin 與 padding 後量測內容，再加回 padding、套用 preferred size，回報包含 margin 的需求大小。**Arrange** 則由父元素分配實際範圍，元素扣除 margin 後依 alignment（Start、Center、End、Stretch）決定自己的 bounds，再將內容區域交給子元素。bounds 改變時會發出 ElementResizeEvent，主視窗即藉此同步 Canvas 的大小。

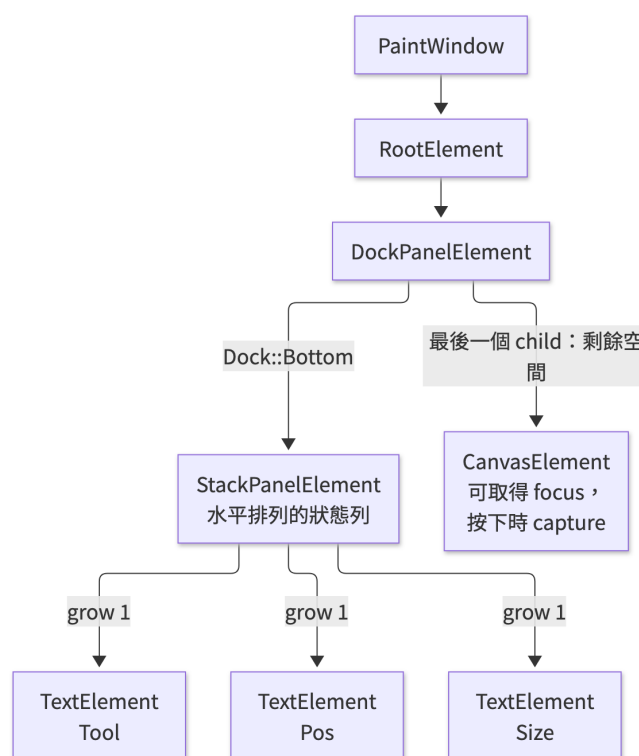


Figure 14: 主繪圖視窗的 Element Tree

兩種容器的配置規則如下：

- **StackPanel**：沿主軸依序排列，量測時主軸方向不限制，讓子元素回報自然大小。剩餘空間依各子元素的 **grow** 權重分配；為避免各自取整後遺失像素，程式以累積權重計算每個子元素的邊界，並把餘數交給最後一個有 **grow** 的子元素。狀態列的三個欄位即以相同權重平分寬度。
- **DockPanel**：子元素依序停靠在剩餘區域的上、下、左、右側，最後一個子元素取得所有剩餘空間。主視窗先將狀態列停靠在底部，再讓 Canvas 填滿其餘區域。

屬性改變時，元素只呼叫 `invalidateLayout()` 標記版面失效；Window 會在下次繪製或 Hit Test 前才重新計算整棵樹，避免同一個事件中多次修改而重複配置。

## A.4 事件分派

所有可接收事件的物件都繼承 `EventTarget`，以事件型別為 **key** 保存 **listener**。`dispatchEvent()` 會先記錄 **target**，再從 **target** 開始呼叫 **listener**；若該事件支援 **bubbling**，便沿著 **parent** 往上傳遞，Element Tree 的根節點之上則是 Window 本身。傳遞過程中 **target** 不變，**current target** 隨所在層級改變，任何一層都可以呼叫 `stopPropagation()` 中止傳遞。

事件的 target 依種類決定，如圖 15：

- **滑鼠事件**：若有元素取得 mouse capture，事件一律送給它；否則以 Hit Test 找到最深層的元素。Canvas、按鈕、色彩區域與 Hue slider 都在按下時 capture、放開時釋放，因此拖曳到元件外仍能收到移動與放開事件。Hover 則永遠依實際位置判斷，不受 capture 影響。
- **鍵盤與文字事件**：送給目前的 focused element，沒有時送給 Window。按下滑鼠時，程式從命中元素沿 parent 往上找最近一個可取得 focus 的元素，並對新舊焦點分別發出 Focus 與 Blur 事件。

FreeGLUT 只回報按下與放開，Click 與 DoubleClick 由 Window 自行合成：按下與放開的距離在 4 px 以內才視為 Click，若與上一次 Click 相隔 300 ms 內且位置相近，則改發 DoubleClick，Polygon 工具即以此結束繪製。

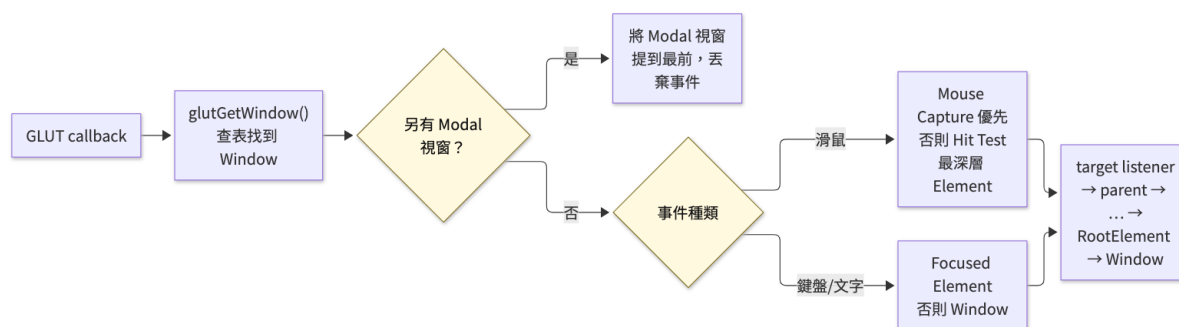


Figure 15: GLUT callback 轉為事件後的分派流程

## A.5 Window、Modal 與生命週期

GLUT callback 是不帶任何物件的 static function，因此 Window 以 GLUT 視窗 ID 為 key 維護一張對照表，callback 觸發時以 glutGetWindow() 查出目前的 Window，再轉交給該物件處理。主繪圖視窗、文字輸入與確認對話框、顏色選擇器，以及 PPM 匯出時使用的視窗，都是 Window 的子類別。

對話框以 Modal 方式開啟：同一時間只記錄一個 Modal 視窗，開啟時先清除所有視窗的按鍵、點擊與 capture 狀態；之後其他視窗收到鍵盤、滑鼠或選單輸入時，會直接丟棄事件並把 Modal 視窗提到最前面。游標進出視窗時清除滑鼠按鍵狀態，元素被移除時也會一併清除指向該子樹的 focus、hover 與 capture，避免之後把事件送到已經不存在的元素。

視窗關閉通常發生在 callback 執行途中，若立刻釋放物件，callback 返回後可能存取到已刪除的記憶體。因此關閉時只將視窗標記為待關閉，由 Application 在 GLUT 的 idle callback 中統一釋放。

## A.6 Menu 與 Shortcut

FreeGLUT 的選單以整數 ID 識別，callback 也只會收到被點選項目的 ID。Menu 將一個 GLUT 選單包裝成物件：建立時記錄所屬視窗，並在全域對照表中登記「選單 ID → Menu」；每個項目則保存「項目 ID → std::function」。子選單同樣是 Menu，由父選單擁有，因此整個選單會形成一棵樹，釋放時一併銷毀。callback 觸發時，先以 `glutGetMenu()` 找到所屬的 Menu，確認沒有被 Modal 視窗擋下後，再執行對應項目的動作。由於 GLUT 以全域狀態記錄「目前的視窗與選單」，新增項目前會暫時切換到自己的視窗與選單，完成後再切回原本的狀態。

快捷鍵由 `ShortcutManager` 管理。每組快捷鍵是一個 key chord，也就是必須同時按住的按鍵與修飾鍵集合；修飾鍵中的 Primary 在 macOS 對應 Command，在其他平台對應 Ctrl，讓同一組綁定能跨平台使用。KeyDown 事件發生時，管理器只考慮包含剛按下按鍵、且所有按鍵都處於按下狀態的 chord，並在多個 chord 同時符合時選擇按鍵數最多的一組，例如按下 Primary+Shift+Z 時執行 Redo，而不是同樣符合的 Primary+Z。按住不放產生的重複事件則會被忽略，避免動作連續觸發。快捷鍵在 Window 層處理，因此鍵盤事件會先送到 focused element，只有元素沒有呼叫 `stopPropagation()` 時才會 bubbling 到 Window 並觸發快捷鍵。主視窗中，選單項目與對應的快捷鍵綁定到同一個函式，避免兩邊的行為不同步。

## B GFNT 自訂點陣字型

### B.1 設計目的與產生方式

GFNT 的目的是在不引入字型解析函式庫的前提下，讓程式能夠顯示中文。FreeGLUT 內建字型只支援 ASCII，而一般的 TTF 字型以曲線描述字形，必須先解析再光柵化，對這次作業而言過於複雜。

因此本專案事先將字型中的字形轉為點陣資料，並以簡單的二進位格式保存。程式執行時只需依 Unicode code point 找到對應的字形，取得其尺寸與定位資訊，再把點陣資料畫到畫面上。

目前隨程式附上兩套字型，使用者可在選單中切換，授權條款隨字型檔一同附上：

- **Cubic 11 (俐方體 11 號)**：以 12 px 轉換，共 10,268 個字元，風格為像素字型。
- **GNU Unifont**：以 16 px 轉換，共 57,071 個字元，涵蓋的字元範圍較廣。

## B.2 檔案結構

GFNT 由三個區段依序組成，所有整數皆為 little-endian，各欄位的位置與大小如圖 16：

- **Header** (16 bytes)：magic 固定為 GFNT，version 為 1；pixelSize 為轉換時的像素大小；ascender 與 descender 分別為基線上方與下方的高度；glyphCount 為字形數量  $N$ 。
- **Glyph Table** ( $18N$  bytes)：每筆記錄一個字元的 code point、bitmap 寬高、bearing、advance，以及 bitmap 在 Bitmap Data 中的位移，並依 code point 嚴格遞增排列。
- **Bitmap Data**：所有字形的 bitmap 依序相接，每個像素以 1 byte 表示 alpha (0–255)，列由上往下存放。

由於 Bitmap Data 緊接在 Glyph Table 之後，起點必為  $16 + 18N$ ，檔案不需要另外記錄各區段的位移。

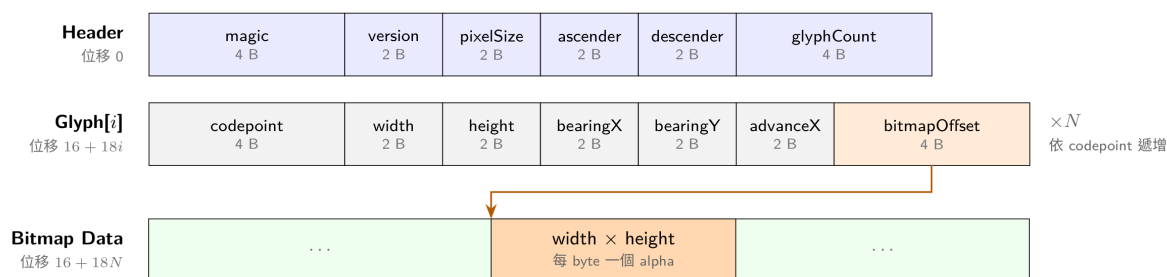


Figure 16: GFNT 的檔案結構；bitmapOffset 為相對於 Bitmap Data 起點的位移

## B.3 Font Metrics 與字形定位

字形以基線對齊。設文字區塊左上角的 Y 座標為  $y_0$ ，則第一行的基線為  $y_b = y_0 + \text{ascender}$ ，每行的行高為  $\text{ascender} + \text{descender}$ 。對 pen 位置  $x_p$  上的一個字形，其 bitmap 左上角為

$$x = x_p + \text{bearingX}, \quad y = y_b - \text{bearingY},$$

畫完後 pen 前進  $\text{advanceX}$ 。 $\text{bearingY}$  是 bitmap 頂端到基線的距離，因此 bitmap 高度大於  $\text{bearingY}$  的部分會延伸到基線以下，如圖 17 中「你」的底部。由於每個字形只儲存實際有墨跡的範圍，空白字元的 bitmap 大小可以是 0，只用  $\text{advanceX}$  推進 pen。



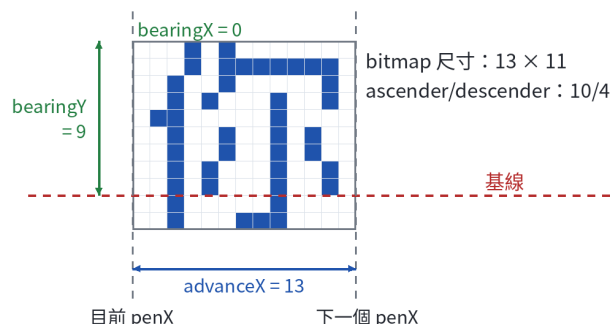


Figure 17: Cubic 11 中「你」(U+4F60) 的字形 metrics；每格代表一個像素

## B.4 載入與驗證

Loader 不直接把檔案內容複製進 C++ struct，而是逐欄位讀取 bytes 並以位移運算組合數值。這樣結果不受執行平台的 endian 與 struct padding 影響，負數也以明確的 two's complement 運算還原。

字型檔在載入時一次完成以下檢查，任何一項失敗都會拒絕載入：

- 檔案至少包含完整的 Header，magic 與 version 正確，pixelSize 不為 0，ascender 與 descender 不為負數。
- glyphCount 所需的 Glyph Table 沒有超出檔案大小，避免依錯誤的數量配置記憶體。
- Glyph Table 依 code point 嚴格遞增，且每個 code point 都是合法的 Unicode 字元（不超過 U+10FFFF，也不是 surrogate）。
- 每個字形的 bitmap 範圍都位於 Bitmap Data 內；比較時以減法改寫  $\text{offset} + w \times h \leq \text{size}$ ，避免加法溢位。

由於所有範圍都在載入時驗證過，繪製時可以直接取用 bitmap，不必每畫一個字都重複檢查邊界。

## B.5 查找與替代字元

Glyph Table 已依 code point 排序，因此以 `std::lower_bound` 進行二分搜尋， $N$  個字形只需  $O(\log N)$  次比較；載入時檢查嚴格遞增，正是為了保證二分搜尋的前提成立。找不到字元時依序改用 U+FFFD 與問號；若字型連這兩者都沒有，則不繪製 bitmap，只以 pixelSize 推進 pen，使後續文字仍維持在正確位置。字形的著色與繪製流程見第 6.4 節。



## C 程式架構

### C.1 系統架構

圖 18 以 C4 model 的 Component 層級描述 Graphtoria。使用者的輸入經 FreeGLUT 轉為 callback，由 GUI Framework（附錄 A）分派給 Canvas；Canvas 將輸入交給目前的 Tool，再依 Tool 回傳的結果修改 Document。Renderers 負責第 4.1–6.4 節的繪製，IO 則負責 GPTD 文件的讀寫與 PPM 匯出。

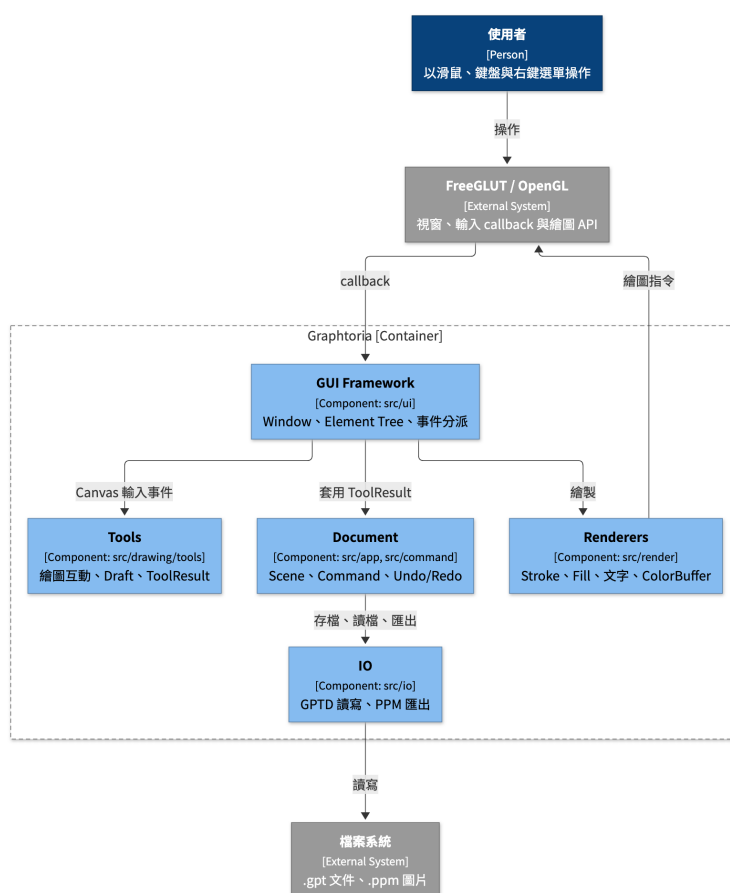


Figure 18: Graphtoria 的 C4 Component 圖

### C.2 資料模型

Document 擁有 Scene 與 CommandHistory，如圖 19。Scene 依繪製順序保存 SceneObject；Shape 依幾何分為單點、兩點決定的圖形與頂點序列，對應第 3.1 節的幾何表示。對 Scene 的每一種修改都包裝成可 `execute()` 與 `undo()` 的 Command，由 CommandHistory 以 `undo` 與 `redo` 兩個堆疊管理。每執行一個 Command 都會產生新的狀態編號，與存檔時的編號比較，即可判斷文件是否有未儲存的變更；因此修改後再 `Undo` 回存檔時的狀態，也會正確視為未修改。

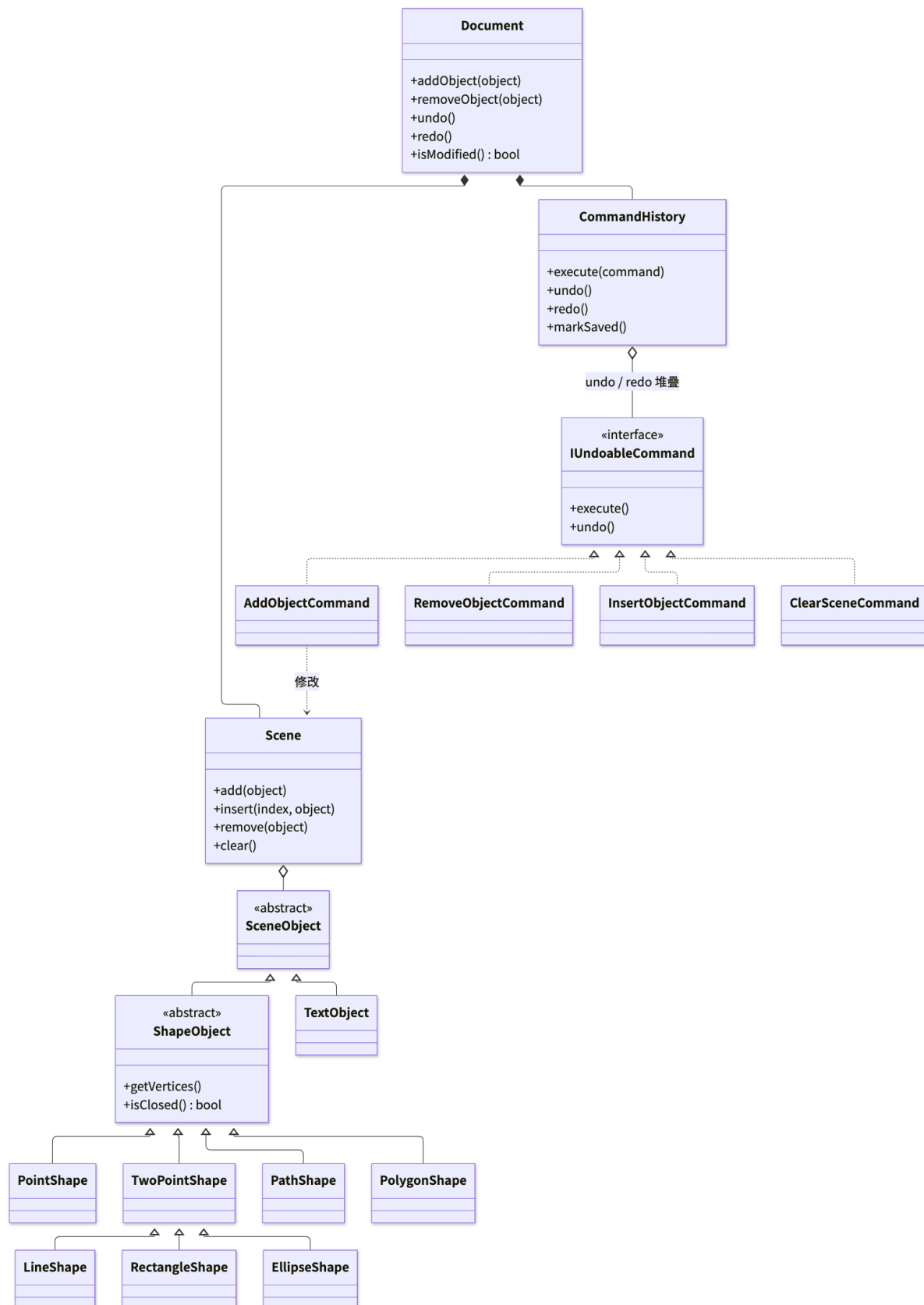


Figure 19: Document、Command 與 Scene 物件的類別圖

### C.3 繪圖工具

所有工具實作 `ICanvasTool`，如圖 20。工具不直接修改 `Document`，而是回傳 `ToolResult`，描述是否需要重畫，以及要新增、刪除物件或開啟文字輸入框，由 `CanvasElement` 統一執行。會建立圖形的工具共用 `CreationTool` 的 Draft 管理：繪製途中的 Draft 只存在於工具內，由 `Renderer` 畫在 `Scene` 上方，完成時才轉為新增物件的動作；拖曳類工具再共用 `DragTool` 的起點、終點與 `Shift` 限制邏輯。

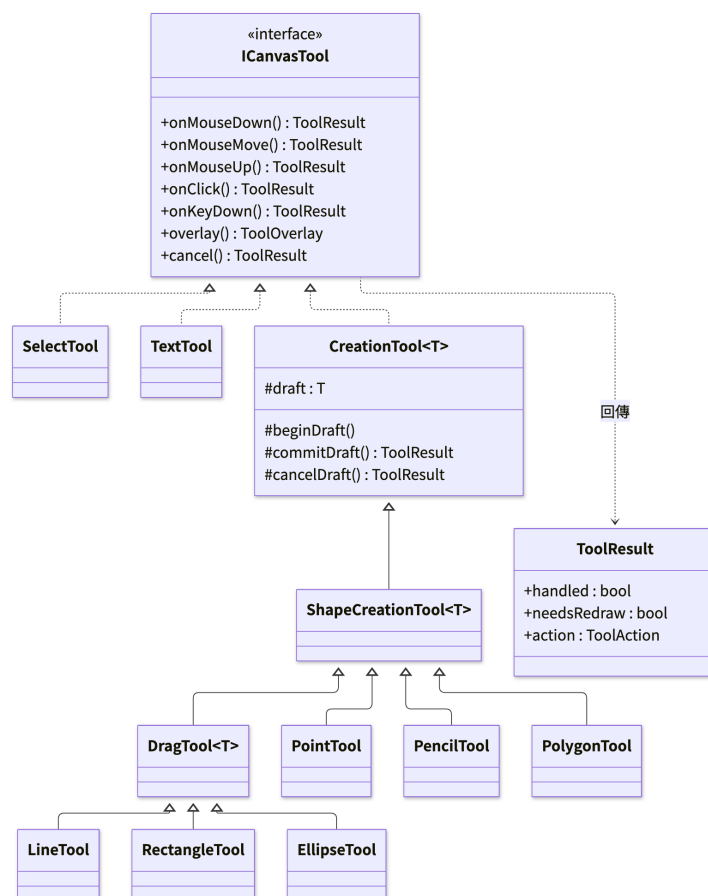


Figure 20: 繪圖工具的類別圖

### C.4 繪製一條直線的流程

圖 21 以 `Line` 工具為例，串起上述元件。按下滑鼠時 `Canvas` 取得 `mouse capture` 並由工具建立 `Draft`；拖曳過程只更新 `Draft` 並要求重畫，不會修改 `Document`；放開滑鼠時工具送出新增物件的動作，`Canvas` 才交給 `Document` 包裝成一筆 `Command` 執行。因此一次拖曳只對應一筆 `Undo` 紀錄，若在拖曳途中按下 `Undo`，則只會取消 `Draft`。

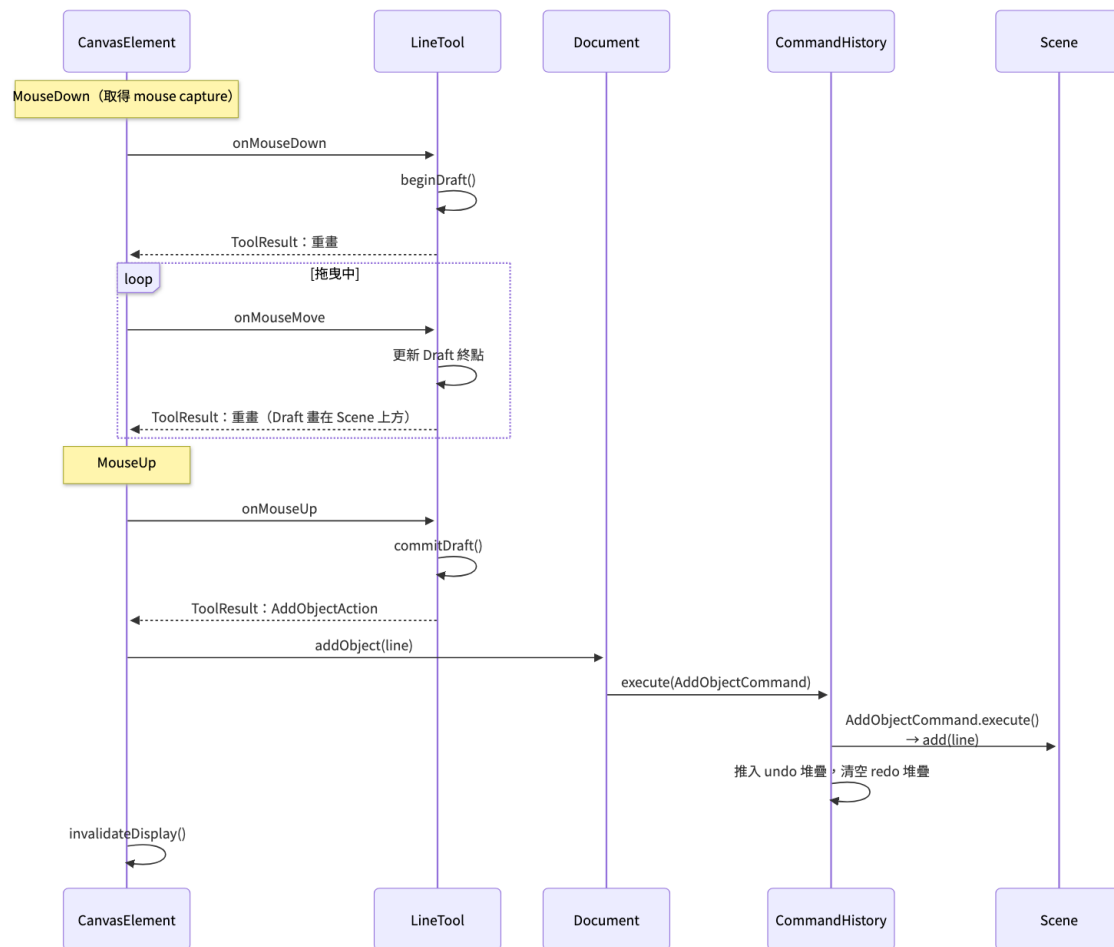


Figure 21: 以 Line 工具繪製一條直線的循序圖